- **Behavior**:
    - Insert shipment into Firestore.
    - Trigger a Google Cloud Function to:
        - Fetch route and traffic from Google Maps Routes API.
        - Fetch weather from the Weather API.
        - Call the ML model (Vertex AI) and update riskScore, delayProbability, eta, and route in Firestore.

2. Get all shipments

- **URL**: GET /api/v1/shipments
- **Response**:

```
json
[
  {
    "id": "shipment_1",
    "origin": { "lat": 19.2, "lng": 72.8 },
    "destination": { "lat": 18.9, "lng": 72.8 },
    "eta": "2026-03-26T10:30:00Z",
    "riskScore": "High" | "Medium" | "Low",
    "delayProbability": 0.85,
    "route": { ... }, // Google Maps route object
    "status": "in_transit" | "delivered"
  }
]
```

3. Get single shipment

- **URL**: GET /api/v1/shipments/:id
- **Response**: Same as above, but single object with extra fields:
    - aiExplanation: string (e.g., "Delay due to traffic + weather").
    - alternateRoutes: array of alternate routes (if available).

4. Trigger risk recomputation (manual / simulation)

- **URL**: POST /api/v1/shipments/:id/simulate
- **Request body**:

```
json
{
  "scenario": "rain_increase" | "traffic_spike",
  "value": 1.5 // factor by which rain/traffic impact is increased
}
```

- **Behavior**:
  - Update shipment's virtual conditions (rain, traffic factor).
  - Recompute delay probability and risk using the ML model.
  - Return updated values without persisting if needed (or write to a simulation field).

---

4. Database Schema (Firestore)

4.1 Collections

- shipments
- users (optional, if authentication is added)
- analytics (optional, for BigQuery sync)

4.2 shipments document structure

Example document:

```
json
{
  "id": "shipment_1",
  "origin": { "lat": 19.2, "lng": 72.8 },
  "destination": { "lat": 18.9, "lng": 72.8 },
  "scheduledTime": "2026-03-26T08:00:00Z",
  "eta": "2026-03-26T10:30:00Z",
  "actualEta": null, // set on arrival
  "priority": "high",
  "status": "in_transit",
  "riskScore": "High",
  "delayProbability": 0.85,
  "route": {
    "overviewPolyline": "...",
```

```
    "legs": [ ... ],
    "duration": 120, // minutes
    "distance": 50.0  // km
  },
  "alternateRoutes": [
    { /* another route object */ }
  ],
  "aiExplanation": "Delay due to traffic + weather",
  "cascadeShipmentIds": ["shipment_2", "shipment_3"],
  "createdAt": "2026-03-25T10:00:00Z",
  "updatedAt": "2026-03-25T10:00:00Z"
}
```

Firestore real-time listeners in the frontend will sync these changes automatically. [web:8]

---

5. External APIs

5.1 Google Maps Routes API

Purpose: Compute routes and traffic-aware ETAs. [web:11]

5.1.1 ComputeRoutes usage

- **Endpoint**: https://routes.googleapis.com/directions/v2:computeRoutes
- **Method**: POST
- **Headers**:
    - Content-Type: application/json
    - X-Goog-Api-Key: ${GOOGLE_MAPS_API_KEY}
- **Request body (simplified)**:

```
json
{
  "origin": { "location": { "latLng": { "latitude": 19.2, "longitude": 72.8 } } },
  "destination": { "location": { "latLng": { "latitude": 18.9, "longitude": 72.8 } } },
  "travelMode": "DRIVE",
  "routingPreference": "TRAFFIC_AWARE",
  "computeAlternativeRoutes": true,
```

```
  "routeModifiers": { "avoidTolls": true }
}
```

- **Response fields used**:
    - routes[0].legs[0].duration (traffic-aware).
    - routes[0].distanceMeters.
    - routes[0].polyline.encodedPolyline (for map display).

5.1.2 Role in the backend

- Used by Cloud Functions to:
    - Compute initial route and ETAs.
    - Compute alternate routes when risk is High. [web:11]

5.2 Weather API (OpenWeather as example)

Purpose: Fetch current weather conditions for shipment coordinates. [web:18]

5.2.1 Current Weather endpoint

- **Endpoint**: https://api.openweathermap.org/data/2.5/weather
- **Method**: GET
- **Query params**:
    - lat, lon
    - units=metric
    - appid=${OPENWEATHER_API_KEY}
- **Response fields used**:
    - weather[0].main (e.g., "Rain").
    - rain.1h (if present).
    - main.temp, wind.speed.

5.2.2 Role in the backend

- Convert main field to a numeric weather condition code (e.g., 1 = clear, 2 = rain).
- Pass weather condition and severity as features to the ML model.

---

6. Google Vertex AI – ML Model

6.1 Model overview

- **Model type** (initial): Logistic Regression or Decision Tree.

- **Goal**: Predict delay probability and risk level from shipment features.

6.2 Feature vector (input)

The backend must send a feature vector like:

```
json
{
  "features": [
    50.0,       // distance (km)
    1.5,        // traffic factor (e.g., 1.0 = normal, 1.5 = heavy)
    2,          // weather condition (1 = clear, 2 = rain, ...)
    8,          // time of day (hour)
    0.4         // historical delay rate for similar route/time
  ]
}
```

6.3 Vertex AI prediction API endpoint

- **Endpoint**: https://<region>-aiplatform.googleapis.com/v1/projects/${PROJECT_ID}/locations/${LOCATION}/endpoints/${ENDPOINT_ID}:predict
- **Method**: POST
- **Headers**:
  - Authorization: Bearer ${ACCESS_TOKEN}
  - Content-Type: application/json
- **Request body**:

```
json
{
  "instances": [
    {
      "features": [50.0, 1.5, 2, 8, 0.4]
    }
  ],
  "parameters": {}
}
```

- **Response** (simplified):

```
json
{
  "predictions": [
    [0.85, 0.45, 0.10] // probabilities for no delay, small delay, large delay
  ]
}
```

- **Backend logic**:
  - Map predictions[0][0] to delayProbability (or combine classes as needed).
  - Convert to risk level using thresholds:
    - Low: < 0.3
    - Medium: 0.3–0.7
    - High: ≥ 0.7

Use this pattern in a Node.js service that Cloud Functions can call. [web:16][web:19]

---

7. Google Cloud Functions (Orchestration)

7.1 Trigger shipment creation

- **Function**: onShipmentCreated
- **Trigger**: Firestore shipments collection onCreate.
- **Behavior**:
  1. Read shipment